样题5

模块一：平台搭建与运维

(一)任务一：大数据平台搭建

1.子任务一：基础环境准备

(1)`ifconfig` 查看ip

`hostnamectl set-hostname master` 配置主机名

`vim /etc/hosts` 打开配置文件

```
ip master
ip slave1
ip slave2
```

`source /etc/hosts` 立即生效

(2)`tar -zxvf /opt/software/jdk-8u191-linux-x64.tar.gz -C /opt/module`

(3)`vim /etc/profile`

```
export JAVA_HOME=/opt/module/jdk
export PATH=$PATH:$JAVA_HOME/bin
```

`source /etc/profile`

(4)`java -version`

(5)`useradd hadoop` 创建用户hadoop

`passwd hadoop` 设置密码

`visudo /etc/sudoers` 配置管理员

```
hadoop ALL=(ALL)    NOPASSWD:ALL
```

`grep 'hadoop' /etc/sudoers`

(6)`systemctl stop firewalld` 关闭防火墙

`systemctl disable firewalld` 禁止开机自启

`systemctl status firewalld` 查看状态

(7)`ssh-keygen -t rsa` 生成密钥

`ssh-copy-id slave1` 复制公钥

`sudo service sshd restart` 重启ssh服务

`ssh slave1(slave2)` ssh连接

2.子任务二：Hadoop 完全分布式安装配置

(1)`tar -zxvf /opt/software/hadoop-3.3.6.tar.gz -C /opt/module`
(2)`mv /opt/module/Hadoop /opt/module/hadoop` 重命名为hadoop
`chown -R hadoop:hadoop /opt/module/hadoop` 修改所属组者
(3)`su - hadoop` 切换用户hadoop
`cd /opt/module/hadoop/etc/hadoop`
hadoop-env.sh

```shell
export JAVA_HOME=/opt/module/jdk
export HADOOP_HOME=/opt/module/hadoop
export HADOOP_CONF_DIR=${HADOOP_HOME}/etc/hadoop
export HADOOP_LOG_DIR=${HADOOP_HOME}/logs
```

core-site.xml

```xml
<property>
   <name>fs.defaultFS</name>
   <value>hdfs://master:9820</value>
</property>
<property>
   <name>hadoop.tmp.dir</name>
   <value>/opt/module/hadoop/tmp</value>
</property>
```

hdfs-site.xml

```xml
<property>
   <name>dfs.replication</name>
   <value>3</value>
</property>
<property>
   <name>dfs.namenode.secondary.http-address</name>
   <value>master:9896</value>
</property>
<property>
   <name>dfs.datanode.data.dir</name>
   <value>/opt/module/hadoop/datanode/data</value>
</property>
```

mapred-site.xml

XML

```xml
<property>
   <name>mapreduce.framework.name</name>
   <value>yarn</value>
</property>
<property>
   <name>mapreduce.jobhistory.address</name>
   <value>master:10020</value>
</property>
<property>
   <name>mapreduce.jobhistory.webapp.address</name>
   <value>master:19888</value>
</property>
```

yarn-site.xml

XML

```xml
<property>
   <name>yarn.resourcemanager.hostname</name>
   <value>master</value>
</property>
<property>
   <name>yarn.nodemanager.aux-services</name>
   <value>mapreduce_shuffle</value>
</property>
<property>
   <name>yarn.log-aggregation-enable</name>
   <value>true</value>
</property>
<property>
   <name>yarn.log.server.url</name>
   <value>http://master:19888/jobhistory/logs</value>
</property>
```

masters

```
master
slave1
slave2
```

workers

```
master
slave1
slave2
```

(4)`scp -r /opt/module/hadoop root@slave1:/opt/module`
`scp -r /opt/module/hadoop root@slave2:/opt/module`
(5)`ssh slave1`
`chown -R hadoop:hadoop /opt/module/hadoop`
(6)`vim /etc/profile`

```
export HADOOP_HOME=/opt/module/hadoop
export PATH=$PATH:$HADOOP_HOME/bin
```

(7)`su - hadoop`
`hdfs namenode -format`
(8)`start-hdfs.sh`
`start-yarn.sh`
`mapred --daemon start historyserver`
(9)`jps`
结果...

(二)任务二：数据库服务器的安装与运维

1.子任务一：MySQL 安装配置

(1)`tar -xvf /opt/software/mysql-5.7.44-1.el7.x86_64.rpm-bundle.tar -C /opt/module`
(2)`rpm -ivh mysql-communitycommon mysql-community-libs mysql-community-libs-compat`
`mysqlcommunity-client mysql-community-server`
(3)`systemctl start mysqld`
`systemctl enable mysqld`
`mysqld --initialize --user=mysql`

2.子任务二：MySQL 运维

(1)`vim /etc/my.cnf`

```
bind-adress = 0.0.0.0
```

(2)`vim /etc/my.cnf` 打开配置文件
`slow_query_log = 1` 启用慢查询日志功能
`long_query_time = 2` 设置为2秒
`systemctl restart mysqld` 重启
(3)`create user 'hadoop'@'%' identified by 'Hadoop@2025';` 创建hadoop用户
(4)`grant select,insert,update on bigdata.* to 'hadoop'@'%';` 授予权限
(5)`create user 'monitor'@'%' identified by '密码';` 创建monitor用户
`grant process,show databases on bigdata.* to 'monitor'@'%';` 授予权限

(6)`mysqldump -u hadoop -p bigdata > bigdata_backup.sql` 使用mysqldump 工具备份 bigdata 数据库
(7)`vim /etc/my.cnf`

```
[mysqld]
character-set-server = utf8mb4
collation-server = utf8mb4_unicode_ci
```

`systemctl resasrt mysqld`
`mysql -u root -p`

```
[mysqld]
show variables like 'character_set_server';
show variables like 'collation_server';
```

(8)

```shell
cat > /.../.../backup.sh << 'EOF'
#!/bin/bash
user="root"
pass="密码"
backup="/opt/backup"
date=$(date +%Y%m%d_%H%M%S)
database="bigdata"
mkdir -p "$backup"

mysqldump -u"$user" -p"$pass" "$database" >
"${backup}/${database}_backup_${date}.sql"

find "$backup" -name "${database}_backup_*.sql" -mtime +7 -delete
EOF

chmod +x /.../.../backup.sh

(crontab -l 2>/dev/null; echo "0 2 * * * /.../.../backup.sh") | crontab -
```

3.子任务三：数据表的创建及维护

(1)`use bigdata;`

```
create table hotel(
   id int,
   hotel_name varchar(n),
   city varchar(n),
   province varchar(n),
   level varchar(n),
   room_num int,
   score double,
   commnent_num varchar(n)
);
```

(2)

```
create table comment(
   id int,
   name varchar(n),
   commentator varchar(n),
   srcore double,
   comment_time datetime,
   content varchar(n)
);
```

(3)`use bigdata;`
`source /opt/../*.sql;`
统计各省份的平均酒店评分,并将评分低于该省份平均分的酒店评分上调0. 5分(评分上限为5分)

```
update hotel as h
set h.score = least(h.score + 0.5,5)
where h.score < (
   select
       avg(h2.score) as 平均分
   from
       hotel h2
   where
       h.province = h2.province
);
```

查找出每个城市评论数最多的酒店及其详细信息

```
select
   h.*
from
   hotel h
where
   cast(h.commnet_num as int) = (
   select
       max(h2.commnet_num)
   from
       hotel as h2
   where
       h.city = h2.city
   );
```

找出所有发表过3条及以上差评（评分小于等于2分）的评论人，并列出这些评论人的评论时间、评论内容及对应的酒店名称，按评论时间降序排序

```
select
   c.commentator,
   c.comment_time,
   c.content,
   c.name
from
   comment as c
where c.commentator in (
   select
       commentator
   from
       comment
   where
       score <=2
   group by
       commentator
   having
       count(*) >= 3
)
order by
   c.comment_time desc;
```

模块二：数据获取与处理

(一)任务一：数据获取与清洗

1.子任务一：数据获取

```
import pandas as pd
data = pd.csv_read('hotel.csv',rnow = 10)
print(data)
```

2.子任务二：使用 Python 进行数据清洗

(1)删除酒店名称为空的记录，并将结果存储为 `cleaned_data_c1_4.csv`，N 为删除的数据条数
(2)将评分数据中的空值替换为 0 分，包括卫生评分、服务评分、设施评分和位置评分，并将结果存储为 `cleaned_data_c2_6.csv`，N 为修改的数据条数
(3)删除评价数为 0 但任一评分项不为 0 的矛盾记录，并将结果存储为 `cleaned_data_c3_4.csv`，N 为删除的数据条数
(4)将装修时间为空的记录设置为数据集中最常见的装修时间（2010-06），并将结果存储为 `cleaned_data_c4_22.csv`，N 为修改的数据条数
(5)删除地址字段完全为空的记录，并将结果存储为 `cleaned_data_c5_5.csv`，N 为删除的数据条数
(6)将房价为 0 或空值的记录删除，并将结果存储为 `cleaned_data_c6_12.csv`，N 为删除的数据条数
(7)统一房间类型的表述，将「标准间」「标准房」统一改为「标准双人房」，并将结果存储为 `cleaned_data_c7_4.csv`，N 为修改的数据条数
(8)删除四项评分（卫生评分、服务评分、设施评分、位置评分）均为 0 的记录，并将结果存储为 `cleaned_data_c8_1.csv`，N 为删除的数据条数
(9)将评分超过 5 分的无效值修正为 5 分，并将结果存储为 `cleaned_data_c9_1.csv`，N 为修改的数据条数
(10)识别并删除完全重复的记录（所有字段值都相同），并将结果存储为 `cleaned_data_c10_0.csv`，N 为删除的数据条数

清洗过程行数变化：原始数据 200 条 → c1 后 196 条 → c2 后 196 条 → c3 后 192 条 → c4 后 192 条 → c5 后 187 条 → c6 后 175 条 → c7 后 175 条 → c8 后 174 条 → c9 后 174 条 → c10 后 174 条。

(二)任务二：数据标注

1.子任务一：酒店价格区间标注

根据房价把酒店分为经济型、中端型和高端型，结果存入 `hotel_price_level.csv`（174 行，列：酒店名称、地区、房价、价格类型）。标注结果分布为：经济型 49 家、中端型 60 家、高端型 65 家，高端型酒店占比最高（37.36%），中端型 34.48%、经济型 28.16%。

2.子任务二：酒店服务质量标注

以（服务评分 + 卫生评分）/ 2 作为服务质量得分并划分等级，结果存入 `hotel_service_level.csv`（174 行，列：酒店名称、服务评分、卫生评分、服务质量得分、服务等级）。标注结果分布为：标准服务 71 家、良好服务 60 家、优质服务 25 家、需改进服务 18 家；全量服务质量得分介于 2.4 分至 4.95 分之间，标准服务与良好服务合计占七成以上，优质服务占比不足 15%，仍有可观的提升空间。

(三)任务三：数据统计

1.子任务一：HDFS 文件操作

(1)`hdfs dfs -mkdir -p /hotel/data` 创建文件夹
`hdfs dfs -mkdir -p /hotel/backup`
(2)`hdfs dfs -put hotel.csv /hotel/data` 上传文件
`hdfs dfs -ls /hotel/data` 验证
(3)`hdfs dfs -chmod 754 /hotel/data` 设置权限
`hdfs dfs -ls -d /hotel/data` 查看权限
(4)`hdfs dfs -cat /hotel/data/hotel.csv | head -n 5` 查看前五行数据内容
(5)`hdfs dfs -cp /hotel/data/hotel.csv /hotel/backup/hotel_backup.csv`

2.子任务二：计算输入文件中的单词数

(1)`hdfs dfs -mkdir /user/hadoop/input`
(2)`hfds dfs -put /var/log/dmesg /user/hadoop/input`
(3)`cd /opt/module/hadoop/share/hadoop/mapreduce`
`hadoop jar hadoop-mapreduce-examples-3.3.6.jar wordcount /var/log/dmesg /user/hadoop/output`
(4)`hdfs dfs -cat /user/hadoop/output/* | head -n 10`

3.子任务三：数独解算器

(1)`cd /opt/module/hadoop/share/hadoop/mapreduce`
`hadoop jar hadoop-mapreduce-examples-*.jar sudoku /提供的文件路径`
(2)看终端

模块三：业务分析与可视化

(一)任务一：数据分析与可视化

1.子任务一：数据分析

(1)各地区酒店数量倒序前两名：北京 21 家、长沙 20 家；深圳同为 20 家，南京 18 家、成都 17 家、武汉 17 家、广州 16 家，上海、杭州、西安各 15 家
(2)不同装修时间的酒店平均设施评分：全部 174 家酒店覆盖 2005-09 至 2025-08 共 114 个月

份，最高的是 2005-09 的 4.60 分，其次 2005-11 的 4.40 分；最低的是 2005-12 的 3.20 分与 2006-02 的 3.00 分，近年月份多在 3.0～4.1 分之间波动
(3)各地区平均房价倒序前三名：杭州 1041.60 元、武汉 1027.24 元、上海 894.40 元；其后依次为广州 888.38 元、南京 853.44 元、西安 852.73 元、深圳 843.85 元、长沙 770.15 元、成都 715.76 元、北京 589.95 元
(4)各房间类型出现频次正序前两名：商务套房 29 家、标准双人房 30 家；标准化后的全部房型为大床房 42 家、家庭房 39 家、行政套房 34 家、标准双人房 30 家、商务套房 29 家
(5)卫生评分和服务评分均高于 4.5 的酒店共 10 家，占全部 174 家酒店的 5.75%

2.子任务二：数据可视化

(1)柱状图展示各个地区的酒店数量，每个柱子代表一个地区，高度代表该地区的酒店数量
(2)条形图比较不同地区酒店的平均卫生评分和服务评分，横轴表示评分，纵轴表示地区，每个地区有两个条形，分别表示卫生和服务评分
(3)雷达图展示任意一间酒店在卫生、服务、设施和位置评分的综合表现，每个轴代表一个评分指标，酒店表现为雷达图上的一个闭合路径
(4)散点图探索房价与评价数量之间的关系，横轴为房价，纵轴为评价数量，每个点代表一家酒店

(二)任务二：业务分析

根据提供的酒店数据集，计算以下指标
(1)平均房价：根据计算，酒店平均价格为837.40元
(2)最受欢迎的房间类型：大床房（评价数合计123288），其后依次为家庭房103177、行政套房77228、商务套房76329、标准双人房70096
(3)评分在前25%的酒店共有的特征：共46家，平均房价1355.80元
房价范围: 163.0 - 2886.0
装修时间范围: 2005-09 - 2025-03
最多的所属公司:同程（11家）
最少的所属公司:飞猪（1家）
在不同地区的最受欢迎的房价类型
上海 行政套房
北京 行政套房
南京 标准双人房
广州 行政套房
成都 标准双人房
杭州 行政套房
武汉 标准双人房
深圳 家庭房
西安 行政套房
长沙 行政套房
酒店的营销策略建议：1.根据用户的需求，优化房间类型的供给。根据数据中顾客最喜欢的是大床房，所以可以适当的将其他较少的房间类型改造。可以在一些旅游的app上对大床房的进行优

惠。
2.对于评分或预订量相对较低的房型，可设计包含早餐、景点门票等增值服务的优惠套餐，以提升吸引力和性价比，促进预订。